

Linguaggi di Programmazione e Modelli Computazionali

Esame del 4 Febbraio 2014

Esercizio 1:

Si mostri un automa a stati finiti deterministico con il numero minimo di stati che riconosce la seguente espressione regolare: $((a^*(ab))b)^*$

Esercizio 2:

Progettare un automa a pila che riconosce il linguaggio sull'alfabeto binario $\{0,1\}$ in cui il numero di occorrenze di 0 è il triplo del numero di occorrenze di 1.

Esercizio 3:

Si consideri il linguaggio delle stringhe binarie del tipo ww (cioè costituite dalla concatenazione di due stringhe identiche). Classificare il linguaggio dicendo se è un linguaggio regolare, libero, ricorsivo, ricorsivamente enumerabile, o nemmeno ricorsivamente enumerabile. Giustificare la risposta.

Esercizio 4:

- * Si consideri il seguente linguaggio sull'alfabeto $\{0,1,2\}$:

$L = \{x2y2z \mid x,y \text{ e } z \text{ sono numeri binari tali che } M_x \text{ riconosce } w_y \text{ in meno di } z \text{ passi}\}$
dove M_x e w_y sono la x -esima macchina di Turing e la y -esima stringa secondo una qualche enumerazione di macchine di Turing e di stringhe, rispettivamente. Dire se L è ricorsivo, ricorsivamente enumerabile, o nemmeno ricorsivamente enumerabile. Giustificare la risposta.

Esercizio 5:

Presentare la definizione di Forma Normale di Chomsky e descrivere (sinteticamente) come si può trasformare una qualsiasi grammatica (che non generi la stringa vuota) in tale forma.

Esercizio 6:

Si consideri la grammatica:

$$E \rightarrow E + F \mid F$$

$$F \rightarrow F * T \mid T$$

$$T \rightarrow (E) \mid x$$

dove E , F e T sono variabili e gli altri sono simboli terminali. Trasformare la grammatica eliminando la ricorsione a sinistra e mostrare la relativa tabella di parsing LL(1).

(No)

Esercizio 7:

Si consideri un sistema concorrente costituito da due colleghi A e B e dalla macchina M usata da A e B in car pooling. A e B sono processi che ciclicamente entrano ed escono da M , mentre M ciclicamente si trova negli stati "sosta" e "viaggia". In tale sistema M può entrare nello stato "viaggia" solamente se entrambi A e B sono presenti in macchina. Descrivere, usando la notazione delle algebre di processo o delle reti di Petri, il comportamento del sistema costituito da A , B ed M e formalizzare in LTL la proprietà sopra descritta.

ESAME LPeMC 4feb2014

Es 1) Regex $\rightarrow$ DFA $((a^* + (ab))b)^*$

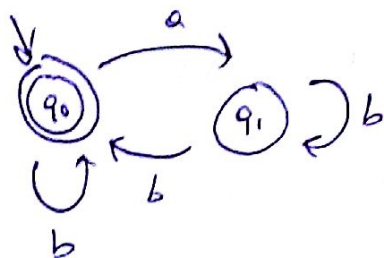
Le parentesi interne sono inutili $((a^* + ab)b)^*$

Cerco di rimuovere un'altra coppia di parentesi $(a^*b + abb)^*$

Nota che abb può essere sempre ottenuto da $(a^*b)^*$
nel seguente modo: $(a^1b)^1(a^0b)^1$

Quindi la regex semplificata è $(a^*b)^*$

Ecco l'automa:



Questo automa è minimo?

Sì, perché $q_0 \in F$ e $q_1 \notin F$
quindi $q_0 \neq q_1$.

Notare che una ipotetica unione degli stati creerebbe
che riconosce $(a+b)^* \neq (a^*b)^*$.



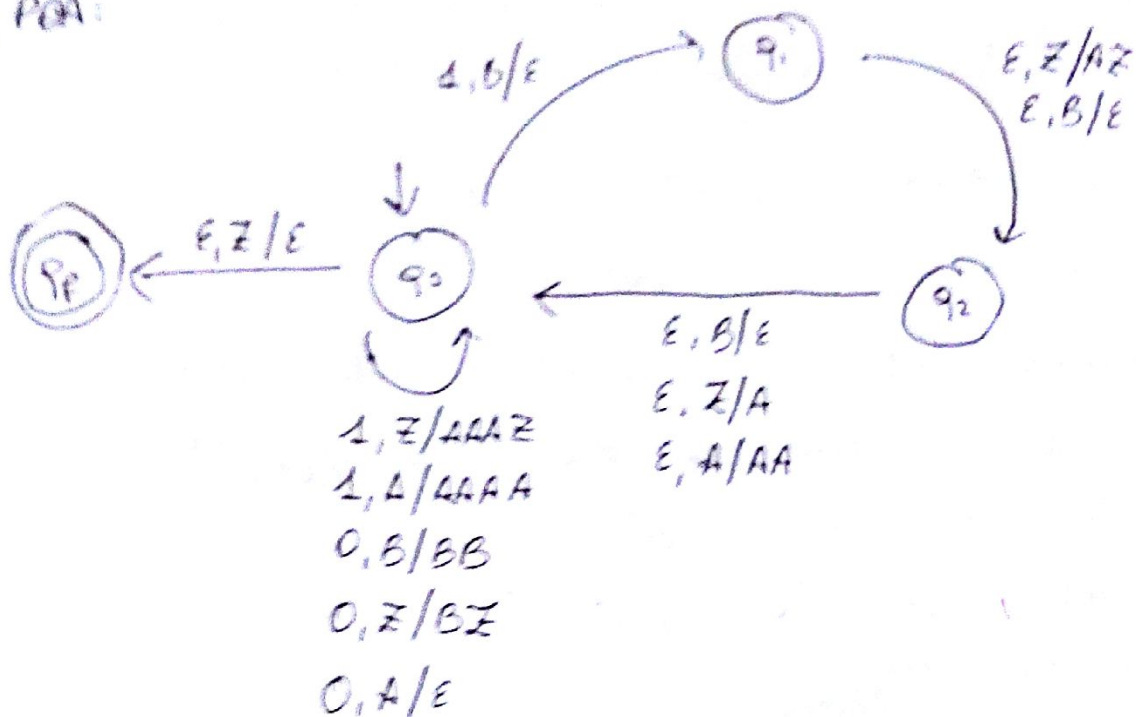
Es. 2) Creazione PDA

Creare PDA P : $L(P) = \{w \in \{0,1\}^* \mid \text{il numero di } 0 \text{ in } w \text{ è il triplo del numero di } 1\}$

Idea:

- Se leggo un 1 e la pila è vuota pusho tre A
- Se leggo un 1 e poppo A, pusho quattro A
- Se leggo uno 0 e poppo A non pusho niente
- Se leggo uno 0 e la pila è vuota pusho B
- Se leggo uno 0 e poppo B, pusho due B.
- Se leggo un 1 e poppo B cerco di poppare altre 2 B,
se non è possibile, pusho A di conseguenza.

es 2, PDA:



N.B. Per comprendere meglio il comportamento, provare a eseguire ~~le~~ i seguenti input:
 1000, 0001, 0100, 00100100

Es. 3) Classificazione linguaggio

$$L = \{z = ww \mid w \in \{0,1\}^*\}$$

Intuizione: Non è libero perché un PDA non può confrontare due tronconi di un input partendo dall'inizio di entrambi. È reversivo perché una macchina di Turing può confrontare due tronconi di una qualunque stringa (le stringhe sono finite per definizione).

Passo 1) Dimostrare che non è libero $\Rightarrow$ Pumping Lemma

$$\exists n: \forall z \in L \text{ con } |z| \geq n, z = z_1 z_2 z_3 z_4 z_5$$

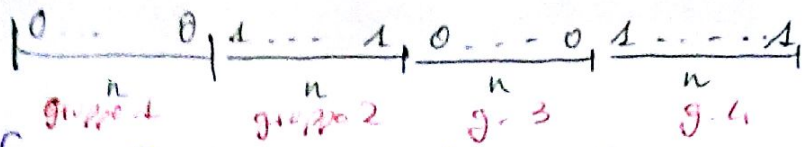
$$\textcircled{I} |z_1 z_3 z_4| \leq n$$

$$\textcircled{II} |z_2 z_4| \geq 1 \Rightarrow \text{almeno uno tra } z_2 \text{ e } z_4 \text{ deve essere } \neq \epsilon$$

$$\textcircled{III} z_i = z_1 (z_2)^i z_3 (z_4)^i z_5 \in L \quad \forall i \in \mathbb{N}$$

Hp assurda L rispetto al PL per i liberi:

$$\text{Considero } z = 0^n 1^n 0^n 1^n \quad |z| = 4n > n$$



→ Se scelgo z_2 tra i primi n zeri allora z_4 può essere

→ $z_4 = \varepsilon$

→ z_4 composta di soli 0 del primo gruppo

→ z_4 composta di 0 e 1 dei gruppi 1 e 2

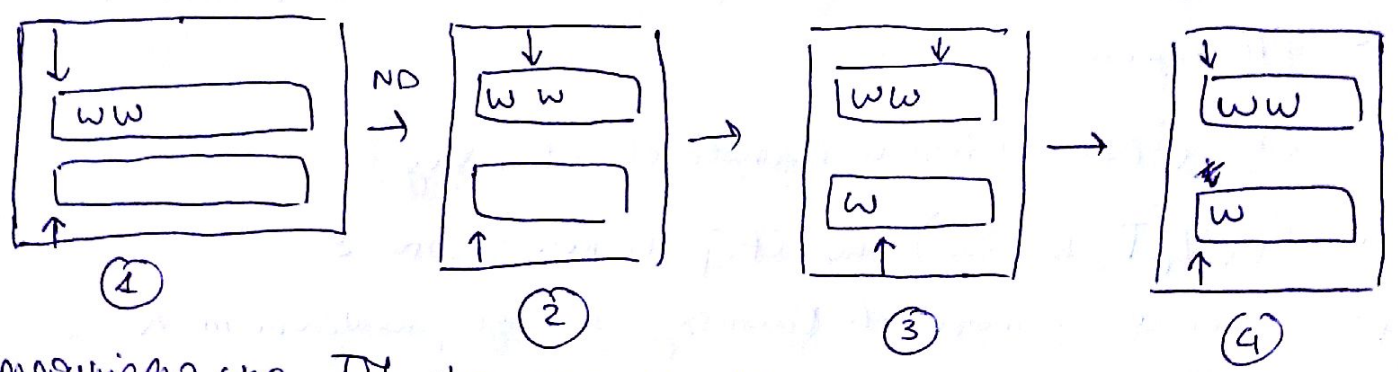
→ z_4 composta di soli 1 del secondo gruppo

In qualunque caso di scelta di z_4 esistono degli i per cui $z_i \neq 1$
 Ad es. per ogni $i \neq 1$ gruppo 1 - gruppo 2 $\neq$ gruppo 3 - gruppo 4

→ Ragionamento analogo per qualunque altra scelta di z_2

$\Rightarrow$ Contraddizione

Passo 2) Progettazione TM per L che termina sempre
 2 nastri

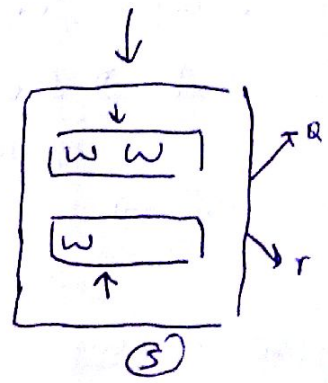


Immaginiamo che TM che inizialmente abbia
 con 2 nastri
 sul primo nastro tutto l'input (1).

Non deterministicamente si sposta a destra di
 alcune porzioni sul primo nastro (2)
 e copia la restante parte del primo nastro
 all'inizio del secondo (3) facendo poi tornare entrambi i
 nastri all'inizio dei rispettivi nastri - (4)

A questo punto confronta i nastri, accettando se alla fine del secondo
 nastro non è stata trovata alcuna differenza con la contro parte
 e riflettendo alle prime coppie diverse (5).

L'istanza delle TM che ha trovato a metà dell'input accetterà se l'input è una stringa
 di L .



Es. 4) Classificazione Linguaggio

$L = \{x2y2z \mid x,y,z \in \{0,1\}^*, \Pi_x \text{ è la x-esima TM, } w_y \text{ è la y-esima stringa e } \Pi_x \text{ riconosce } w_y \text{ in meno di } z \text{ passi}\}$.

Lo è ricorsivo poiché è possibile costruire una TM che su qualunque triple (x,y,z) .

Progettazione:

Consideriamo una UTM modificata con 2 nastri in più:

→ uno per memorizzare z

→ uno per tenere il conto, durante la simulazione di Π_x sull'input w_y , del numero di passi eseguiti.

La UTM simula Π_x e, ad ogni passo incrementa il conteggio.

Se la simulazione termina prima che il conteggio raggiunga z , si accetta. Se invece il conteggio raggiunge z , la simulazione termina e la TM rifiuta.

Es. 5) CNF (Forma normale di Chomsky)

Sia $G = (V, T, R, S_0)$ una CFG che non genera ϵ .

G è in forma normale di Chomsky se ogni produzione in R è nelle forme

$A \rightarrow BC$

oppure

$A \rightarrow a$

con $A, B, C \in V$ e $a \in T$

Se G non è in CNF è possibile applicare delle trasformazioni che generano una grammatica equivalente in CNF. Nell'ordine:

I) Si eliminano le ϵ -produzioni ($X \rightarrow \epsilon$) $X \in V$

II) Si eliminano le produzioni-unità ($A \rightarrow B$) $A, B \in V$

III) Si eliminano i simboli inutili (ovvero non raggiungibili e/o non generanti)

IV) Si introducono nuove variabili per avere tutte le produzioni nelle forme $A \rightarrow BC$
 $A \rightarrow a$

Es 6) Verificare grammatica LL1

$$E \rightarrow (E) + F \mid F$$

$$F \rightarrow F * T \mid T$$

$$T \rightarrow (E) \mid x$$

~~La grammatica non rispetta le produzioni
che simboli inutili che le produzioni
non è fattorizzata e bisogna decidere
l'ordine di calcolo.~~

Bisogna rimuovere le ricorsione e fattorizzare prima di procedere.

	first	follow
$E \rightarrow FE'$	$x, ($	$\$,)$
$E' \rightarrow +FE' \mid \epsilon$	$+, \epsilon$	$\$,)$
$F \rightarrow TF'$	$x, ($	$+, \$,)$
$F' \rightarrow *TF' \mid \epsilon$	$\epsilon, *$	$+, \$,)$
$T \rightarrow (E) \mid x$	$x, ($	$*, +, \$,)$

(non c'è niente da fattorizzare!)

	(	)	*	+	x	\$
E	FE'				FE'	
E'		ϵ		+FE'		ϵ
F	TF'				TF'	
F'		ϵ	*TF	ϵ		ϵ
T	(E)				x	

Dato che nelle tabelle LL1 non ci sono conflitti, la grammatica è LL1